

Отчёта по лабораторной работе 5

Имитационное моделирование

Нджову Нелиа

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Сети Петри	7
3.2	Задача «Обедающие философы»	8
4	Выполнение лабораторной работы	9
4.1	1. Код модели	9
4.2	2. Базовые эксперименты	10
4.3	Базовые эксперименты	13
4.4	Базовые эксперименты	18
4.5	3. Анимация процесса	21
4.6	4. Итоговый отчёт	22
4.7	Итоговый отчёт	24
4.8	Итоговый отчёт	28
5	Выводы	33

Список иллюстраций

4.1	Код модели	9
4.2	Код базовых экспериментов	10
4.3	classic_simulation.png	11
4.4	arbiter_simulation.png	12
4.5	файл jupyter	13
4.6	изменения количества философов	17
4.7	файл jupyter	17
4.8	Код Анимации процесса	22
4.9	Код Итогового отчёта	23
4.10	файл jupyter	23
4.11	Код Итогового отчёта(N=2)	27
4.12	Код Итогового отчёта(N=10)	27
4.13	файл jupyter	28

Список таблиц

1 Цель работы

Цель этой лабораторной работы - моделирования Аппарат сетей петри.

2 Задание

1. Код модели
2. Базовые эксперименты
3. Анимация процесса
4. Итоговый отчёт

3 Теоретическое введение

3.1 Сети Петри

Сеть Петри есть математический аппарат для моделирования дискретных систем, в которых процессы происходят параллельно и асинхронно. Она позволяет выявлять такие проблемы, как взаимная блокировка (deadlock). Графически она представляется как двудольный ориентированный граф двух типов вершин:

- Позиции (Places) суть пассивные элементы, описывающие состояние системы. Внутри позиции могут находиться фишки (tokens, неотрицательное целое число, указывающее на количество ресурсов)
- Переходы (Transitions) суть активные элементы, описывающие события или действия системы. Они могут срабатывать, изменяя состояние модели.
- Дуги соединяют позиции с переходами и показывают, какие ресурсы нужны для события и какие появляются после него.

Правило срабатывания перехода: Переход может сработать, если каждая его входная позиция содержит хотя бы одну фишку. При срабатывании из входных позиций фишки удаляются, а в выходные добавляются. Этот процесс меняет маркировку сети, то есть её текущее состояние.

Сети Петри бывают детерминированными (описываются дифференциальными уравнениями) и стохастическими (используют алгоритм Гиллеспи, где время и выбор перехода определяются случайным образом).

3.2 Задача «Обедающие философы»

Это классическая задача, иллюстрирующая проблему синхронизации в параллельных системах.

Постановка задачи: За круглым столом сидят N философов (обычно 5). Перед каждым тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка. Всего вилок столько же, сколько философов.

Правила: Чтобы поест, философу нужны две вилки — левая и правая. Если вилок нет, философ ждёт. Поев, философ кладёт вилки обратно и возвращается к размышлениям.

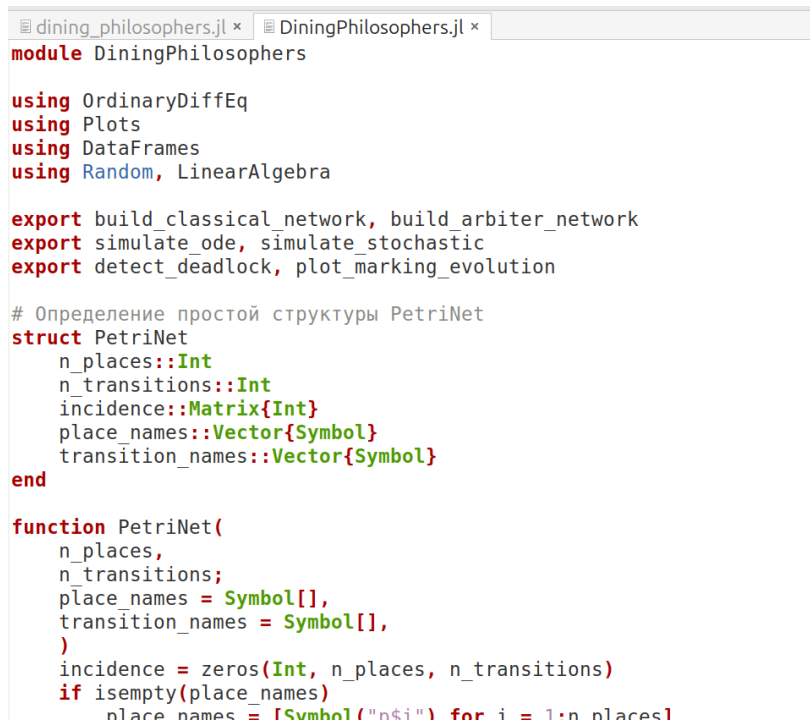
Проблема взаимной блокировки (deadlock): Если все философы одновременно возьмут левую вилку, то каждый будет держать одну вилку и ждать правую, которая занята соседом. В результате никто не может есть — система замирает. Это и есть deadlock.

Решение (арбитр): Чтобы предотвратить deadlock, вводится арбитр (официант), который разрешает есть не более чем $N-1$ философам одновременно. Это гарантирует, что хотя бы один философ всегда сможет взять обе вилки, и система не блокируется.

4 Выполнение лабораторной работы

4.1 1. Код модели

Я сначала создала файл `src/DiningPhilosophers.jl`. Потом скопировала заданный скрипт в него. Скрипт содержит основные функции модели. Я запускала и все работало хорошо (рис.1)



```
module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution

# Определение простой структуры PetriNet
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end

function PetriNet(
    n_places,
    n_transitions;
    place_names = Symbol[],
    transition_names = Symbol[],
)
    incidence = zeros{Int, n_places, n_transitions}
    if isempty(place_names)
        place_names = [Symbol("p"&i) for i = 1:n_places]
```

Рисунок 4.1: Код модели

4.2 2. Базовые эксперименты

Я создала файл `scripts/dining_philosophers.jl`. Потом скопировала заданный скрипт в него. Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock). И модифицированная модель с арбитром, которая должна предотвращать deadlock. Я преобразовала код в литературный стиль и запускала.(рис.2)

```
dining_philosophers.jl x DiningPhilosophers.jl x
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

N = 5
tmax = 50.0

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))
```

Рисунок 4.2: Код базовых экспериментов

После запуск, я получила две графики `classic_simulation.png` и `arbiter_simulation.png`. Мы увидим четыре панели (Think, Hungry, Eat, Fork) с динамикой для каждого философа. В классическом симуляторе произошла взаимоблокировка, но в режиме арбитра этого не произошло(рис 3 и 4)

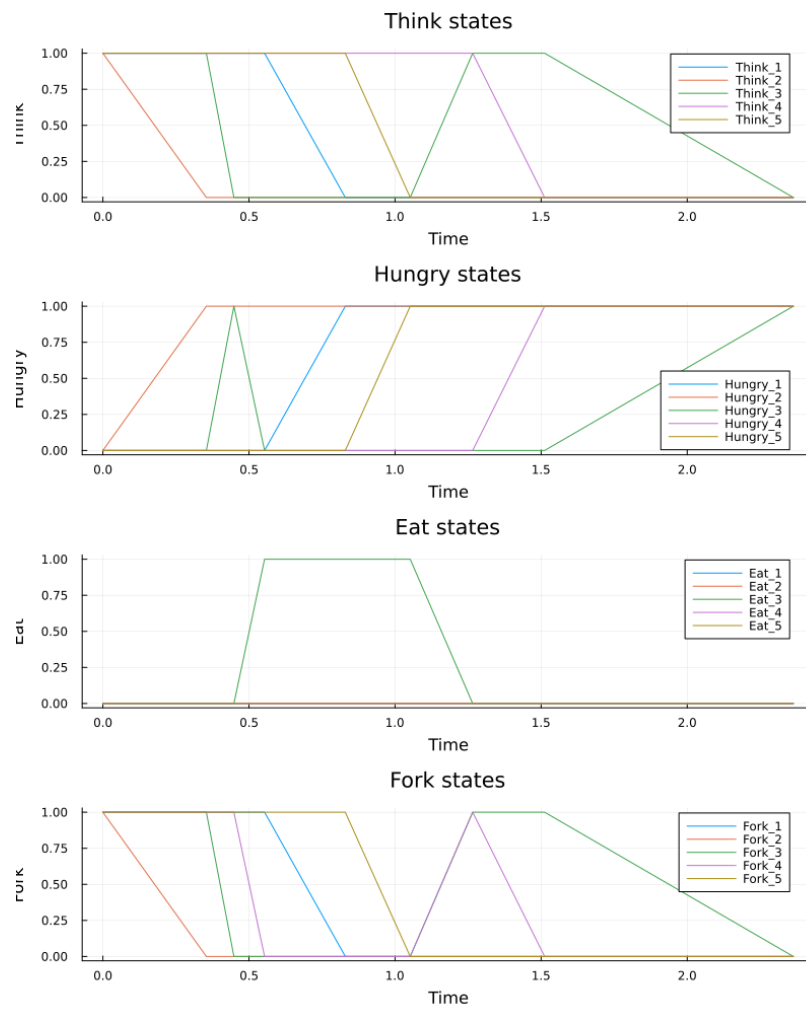


Рисунок 4.3: classic_simulation.png

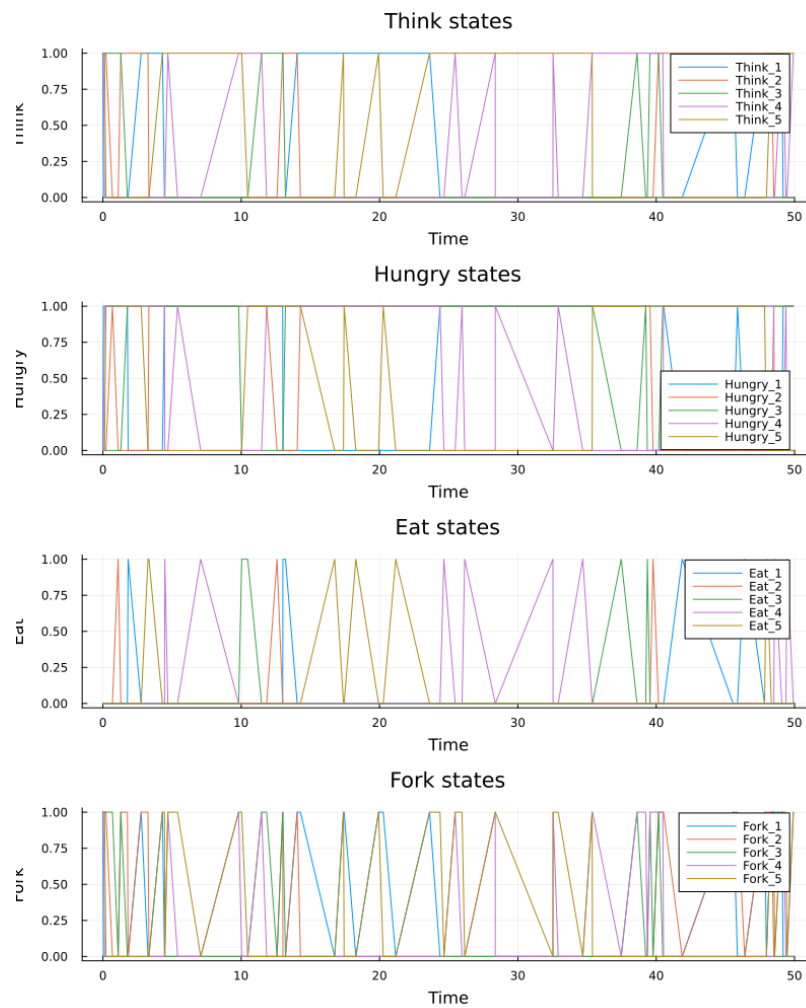
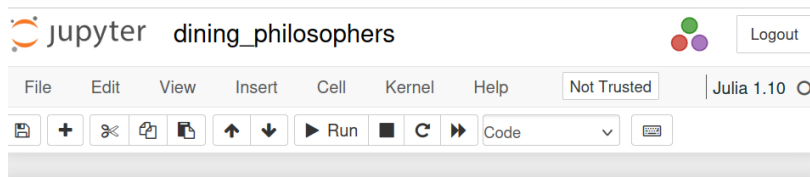


Рисунок 4.4: arbiter_simulation.png

Я генерировала файл juryter, запускала все ячейки и всё работал хорошо.(рис.5)



Базовые эксперименты

Зачем нужен Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock) и модифицированная модель с арбитром, которая должна предотвращать deadlock.

```
In [1]: using DrWatson
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots
```

Создаем классическую сеть Петри для N=5 философов с помощью `build_classical_network`. Запускаем стохастическую симуляцию (алгоритм Гиллеспи) на время `tmax = 50.0`.

Рисунок 4.5: файл `jupyter`

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/dining_philosophers.qmd`

4.3 Базовые эксперименты

Зачем нужен Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock) и модифицированная модель с арбитром, которая должна предотвращать deadlock.

```
using DrWatson
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`  
  | (given dir: /home/nelianjovu)  
  └ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84  
WARNING: replacing module DiningPhilosophers.  
WARNING: using DiningPhilosophers.build_classical_network in module Main conflicts with  
WARNING: using DiningPhilosophers.build_arbiter_network in module Main conflicts with a  
WARNING: using DiningPhilosophers.simulate_stochastic in module Main conflicts with an e  
WARNING: using DiningPhilosophers.detect_deadlock in module Main conflicts with an exist  
WARNING: using DiningPhilosophers.plot_marking_evolution in module Main conflicts with a
```

Создаем классическую сеть Петри для $N=5$ философов с помощью `build_classical_network`. Запускаем стохастическую симуляцию (алгоритм Гиллеспи) на время `tmax = 50.0`.

```
N = 5  
tmax = 50.0  
  
println("=== Классическая сеть (без арбитра) ===")  
net_classic, u0_classic, _ = build_classical_network(N)  
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
```

=== Классическая сеть (без арбитра) ===

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0
2	0.0875217	1.0	0.0	1.0	1.0	1.0	0.0	1.0
3	0.324143	1.0	0.0	1.0	1.0	0.0	0.0	1.0
4	0.535789	1.0	0.0	1.0	1.0	0.0	0.0	0.0
5	0.591992	1.0	0.0	1.0	1.0	0.0	0.0	0.0
6	0.628374	1.0	0.0	1.0	1.0	1.0	0.0	0.0
7	0.966648	1.0	0.0	1.0	0.0	1.0	0.0	0.0
8	1.05722	1.0	0.0	1.0	0.0	1.0	0.0	0.0
9	1.07017	0.0	0.0	1.0	0.0	1.0	1.0	0.0
10	1.11142	0.0	1.0	1.0	0.0	1.0	1.0	0.0
11	2.1125	0.0	1.0	1.0	1.0	1.0	1.0	0.0
12	2.69011	0.0	1.0	1.0	1.0	0.0	1.0	0.0
13	2.82141	0.0	1.0	0.0	1.0	0.0	1.0	0.0
14	3.25714	0.0	0.0	0.0	1.0	0.0	1.0	1.0
15	3.46006	0.0	0.0	0.0	1.0	0.0	1.0	1.0
16	4.26298	0.0	0.0	1.0	1.0	0.0	1.0	1.0
17	4.3116	0.0	0.0	1.0	1.0	0.0	1.0	0.0
18	4.69192	0.0	0.0	1.0	0.0	0.0	1.0	0.0
19	6.80485	0.0	1.0	1.0	0.0	0.0	1.0	0.0
20	7.07096	0.0	1.0	0.0	0.0	0.0	1.0	0.0
21	8.65562	0.0	0.0	0.0	0.0	0.0	1.0	1.0

Сохраняем полную историю маркировок в CSV-файл (dining_classic.csv).

```
CSV.write(datadir("dining_classic.csv"), df_classic)
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab05/project/data/dining_classic.csv"
```

Анализируем последнее состояние на предмет deadlock с помощью detect_deadlock и выводит результат в консоль.

```
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
```

Deadlock обнаружен: true

Строим графики эволюции маркировки по всем позициям (Think, Hungry, Eat, Fork) и сохраняет их как classic_simulation.png.

```
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab05/project/plots/classic_simulation.png"
```

Повторяем для сети с арбитром (build_arbiter_network), сохраняя результаты в dining_arbiter.csv и arbiter_simulation.png.

```
println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))
```

=== Сеть с арбитром ===

Deadlock обнаружен: false

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab05/project/plots/arbiter_simulation.png"

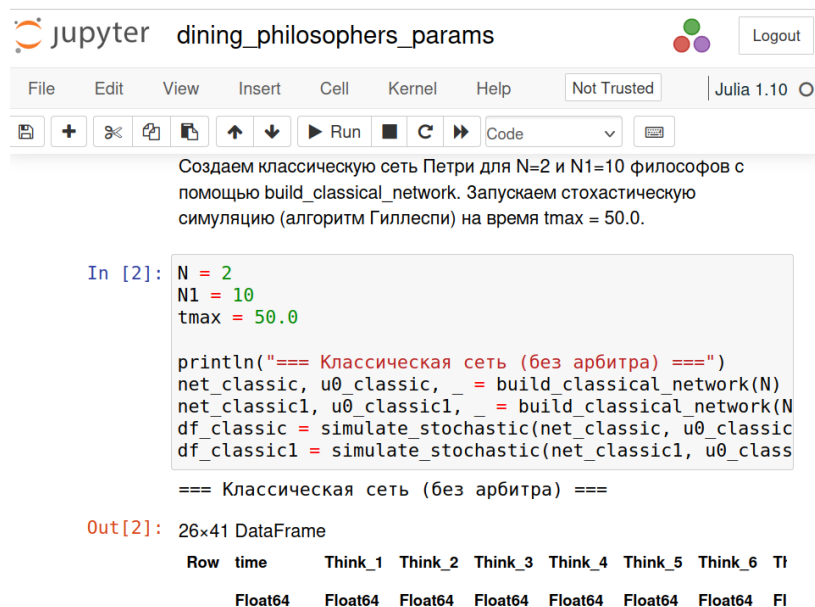
Я изменила количество философов в эксперименте, я использовала $N = 2$ и $N = 10$. И в обоих экспериментах тупик возник в классическом моделировании, но не в произвольном моделировании(рис.6)

```
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab05/project$ julia --project=. scripts/dining_philosophers_params.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен(N=2): true
Deadlock обнаружен(N=10): true

=== Сеть с арбитром ===
Deadlock обнаружен(N=2): false
Deadlock обнаружен(N=10): false
```

Рисунок 4.6: изменения количества философов

Я генерировала файл jupyter, запускала все ячейки и всё работало хорошо.(рис.7)



The screenshot shows a Jupyter Notebook titled "dining_philosophers_params". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Help), a toolbar with icons for file operations and execution, and a status bar indicating "Not Trusted" and "Julia 1.10".

The code cell (In [2]:) contains the following Julia code:

```
N = 2
N1 = 10
tmax = 50.0

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
net_classic1, u0_classic1, _ = build_classical_network(N1)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
df_classic1 = simulate_stochastic(net_classic1, u0_classic1, tmax)

println("=== Классическая сеть (без арбитра) ===")
```

The output (Out [2]:) shows a 26x41 DataFrame with columns: Row, time, Think_1, Think_2, Think_3, Think_4, Think_5, Think_6, and Time. The data type for the first seven columns is Float64, and for the last column (Time), it is Float64.

Рисунок 4.7: файл jupyter

Ниже приведен код, результаты расчетов и графики для кода в файле scripts/dining_philosophers_params.qmd

4.4 Базовые эксперименты

Зачем нужен Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock) и модифицированная модель с арбитром, которая должна предотвращать deadlock.

```
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
WARNING: replacing module DiningPhilosophers.
WARNING: using DiningPhilosophers.build_classical_network in module Main conflicts with ar
WARNING: using DiningPhilosophers.build_arbiter_network in module Main conflicts with ar
WARNING: using DiningPhilosophers.simulate_stochastic in module Main conflicts with an e
WARNING: using DiningPhilosophers.detect_deadlock in module Main conflicts with an exist
WARNING: using DiningPhilosophers.plot_marking_evolution in module Main conflicts with a
```

Создаем классическую сеть Петри для $N=2$ и $N1=10$ философов с помощью `build_classical_network`. Запускаем стохастическую симуляцию (алгоритм Гиллеспи) на время $t_{max} = 50.0$.

```
N = 2
N1 = 10
tmax = 50.0
```

```
println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
net_classic1, u0_classic1, _ = build_classical_network(N1)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
df_classic1 = simulate_stochastic(net_classic1, u0_classic1, tmax)
```

=== Классическая сеть (без арбитра) ===

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Think_6	Think_7	T
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	I
1	0.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	
2	0.0427495	1.0	1.0	1.0	1.0	1.0	1.0	1.0	
3	0.0723963	1.0	1.0	1.0	1.0	0.0	1.0	1.0	
4	0.1111753	1.0	1.0	1.0	1.0	0.0	1.0	1.0	
5	0.206046	1.0	1.0	0.0	1.0	0.0	1.0	1.0	
6	0.23235	1.0	1.0	0.0	1.0	0.0	0.0	1.0	
7	0.63024	1.0	1.0	0.0	1.0	0.0	0.0	1.0	
8	0.689216	1.0	1.0	0.0	0.0	0.0	0.0	1.0	
9	0.782221	1.0	1.0	0.0	0.0	0.0	0.0	0.0	
10	0.894808	1.0	0.0	0.0	0.0	0.0	0.0	0.0	
11	1.52887	1.0	0.0	0.0	0.0	0.0	0.0	0.0	
12	3.19943	1.0	0.0	0.0	0.0	0.0	0.0	0.0	
13	3.237	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
14	3.92789	0.0	0.0	0.0	0.0	0.0	0.0	0.0	

Сохраняем полную историю маркировок в CSV-файл (dining_classic.csv).

```
CSV.write(datadir("dining_classic(N=2).csv"), df_classic)
CSV.write(datadir("dining_classic(N=10).csv"), df_classic1)
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab05/project/data/dining_classic(N=10).csv"
```

Анализируем последнее состояние на предмет deadlock с помощью detect_deadlock и выводит результат в консоль.

```
dead = detect_deadlock(df_classic, net_classic)
dead1 = detect_deadlock(df_classic1, net_classic1)
println("Deadlock обнаружен(N=2): $dead")
println("Deadlock обнаружен(N=10): $dead1")
```

```
Deadlock обнаружен(N=2): true
```

```
Deadlock обнаружен(N=10): true
```

Строим графики эволюции маркировки по всем позициям (Think, Hungry, Eat, Fork) и сохраняет их как classic_simulation.png.

```
plot_classic = plot_marking_evolution(df_classic, N)
plot_classic1 = plot_marking_evolution(df_classic1, N1)
savefig(plotsdir("classic_simulation(N=2).png"))
savefig(plotsdir("classic_simulation(N=10).png"))
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab05/project/plots/classic_simulation(N=10).png"
```

Повторяем для сети с арбитром (build_arbiter_network), сохраняя результаты в dining_arbiter.csv и arbiter_simulation.png.

```
println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
```

```

CSV.write(datadir("dining_arbiter(N=2).csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен(N=2): $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation(N=2).png"))

net_arb1, u0_arb1, _ = build_arbiter_network(N1)
df_arb1 = simulate_stochastic(net_arb1, u0_arb1, tmax)
CSV.write(datadir("dining_arbiter(N=10).csv"), df_arb1)
dead_arb1 = detect_deadlock(df_arb1, net_arb1)
println("Deadlock обнаружен(N=10): $dead_arb1")
plot_arb1 = plot_marking_evolution(df_arb1, N1)
savefig(plotsdir("arbiter_simulation(N=10).png"))

```

=== Сеть с арбитром ===

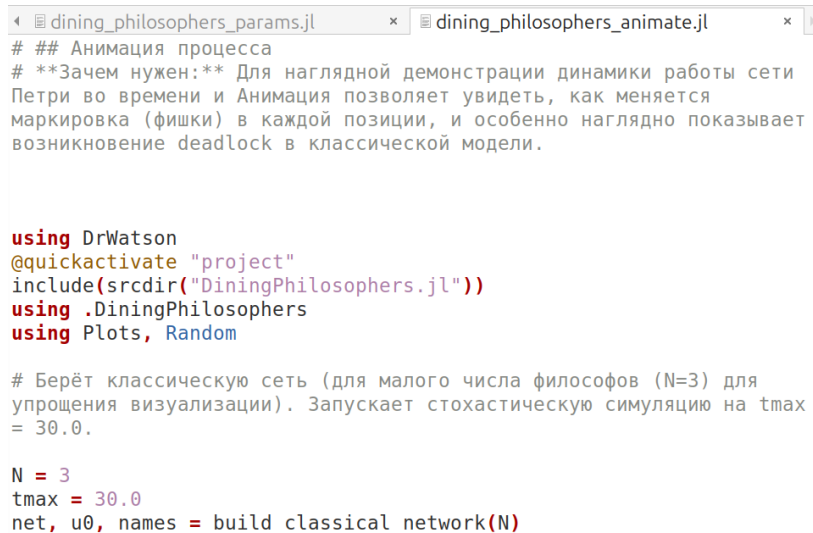
Deadlock обнаружен(N=2): false

Deadlock обнаружен(N=10): false

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab05/project/plots/arbiter_simulation(N=10).png"

4.5 3. Анимация процесса

Я создала файл `scripts/dining_philosophers_animation.jl`. Потом скопировала заданный скрипт в него. Скрипт используется для наглядной демонстрации динамики работы сети Петри во времени. Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно показывает возникновение deadlock в классической модели(рис.8)



```

dining_philosophers_params.jl x dining_philosophers_animate.jl x
# ## Анимация процесса
# **Зачем нужен:** Для наглядной демонстрации динамики работы сети
# Петри во времени и Анимация позволяет увидеть, как меняется
# маркировка (фишки) в каждой позиции, и особенно наглядно показывает
# возникновение deadlock в классической модели.

using DrWatson
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random

# Берёт классическую сеть (для малого числа философов (N=3) для
# упрощения визуализации). Запускает стохастическую симуляцию на tmax
# = 30.0.

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

```

Рисунок 4.8: Код Анимации процесса

4.6 4. Итоговый отчёт

Я создала файл `scripts/dining_philosophers_report.jl`. Потом скопировала заданный скрипт в него. Скрипт используется для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (`Eat_i`). Скрипт генерирует сводный график.(рис.9)

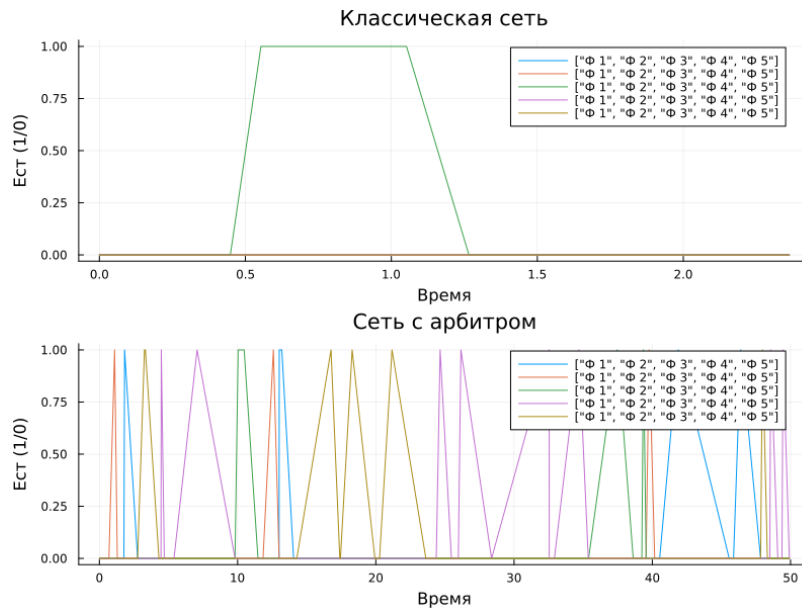


Рисунок 4.9: Код Итогового отчёта

Я преобразовала код в литературный стиль, генерировала файл jupyter и выполнила все ячейки(рис.10)

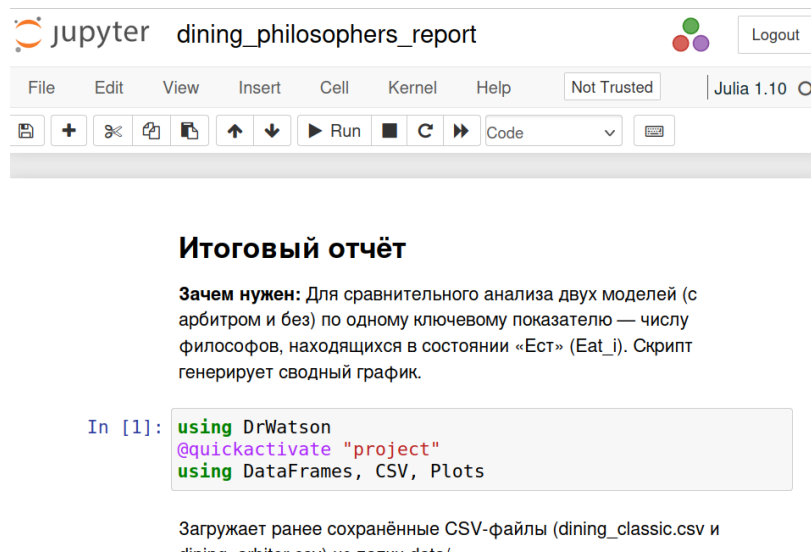


Рисунок 4.10: файл jupyter

Ниже приведен код, результаты расчетов и графики для кода в файле scripts/scripts/dining_philosophers_report.qmd

4.7 Итоговый отчёт

Зачем нужен: Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i). Скрипт генерирует сводный график.

```
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
gr(fmt=:png)
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
  └ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

```
Plots.GRBackend()
```

Загружает ранее сохранённые CSV-файлы (dining_classic.csv и dining_arbiter.csv) из папки data/.

```
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5
```

5

Столбцы для состояния «Ест»

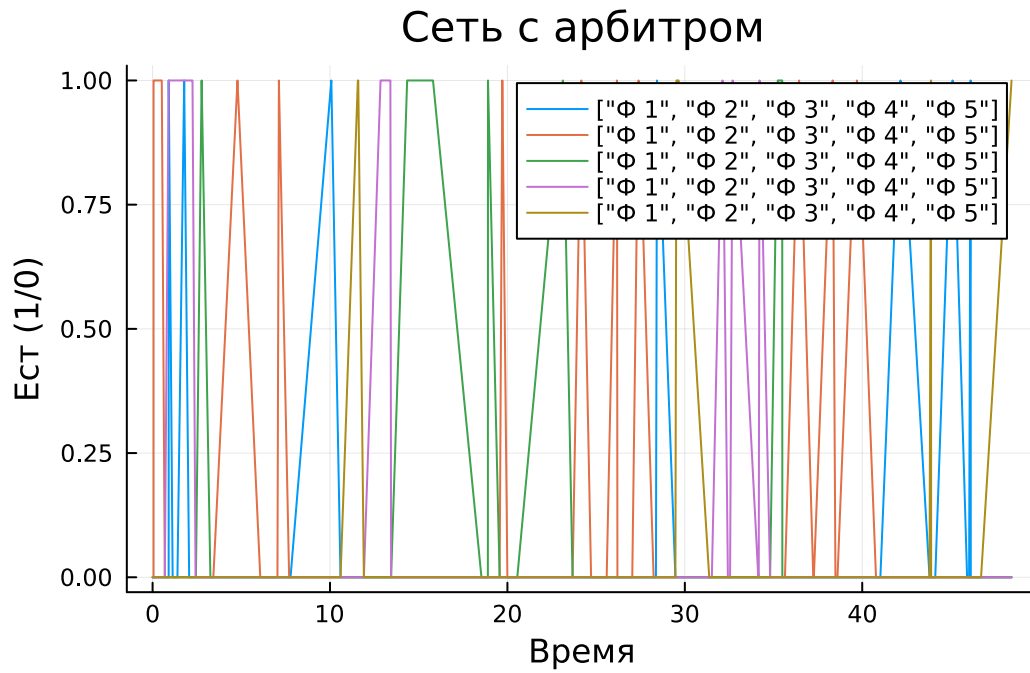
```
eat_cols = [Symbol("Eat_$i") for i = 1:N]
```

```
5-element Vector{Symbol}:
 :Eat_1
```


:Eat_2
:Eat_3
:Eat_4
:Eat_5

Строит два временных графика (один под другим): динамику «едающих» философов для классической сети и для сети с арбитром.

```
p1 = plot(  
    df_classic.time,  
    Matrix(df_classic[:, eat_cols]),  
    label = [" $\Phi_i$ " for i = 1:N],  
    xlabel = "Время",  
    ylabel = "Ест (1/0)",  
    title = "Классическая сеть",  
)  
  
p2 = plot(  
    df_arbiter.time,  
    Matrix(df_arbiter[:, eat_cols]),  
    label = [" $\Phi_i$ " for i = 1:N],  
    xlabel = "Время",  
    ylabel = "Ест (1/0)",  
    title = "Сеть с арбитром",  
)
```



Сохраняет объединённый график как final_report.png в папку plots/.

```
p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report.png"))
println("Отчёт сохранён в plots/final_report.png")
```

Отчёт сохранён в plots/final_report.png

Я изменила скрипт, чтобы он работал при $N = 2$ и $N = 10$ (рис. 11 и 12)

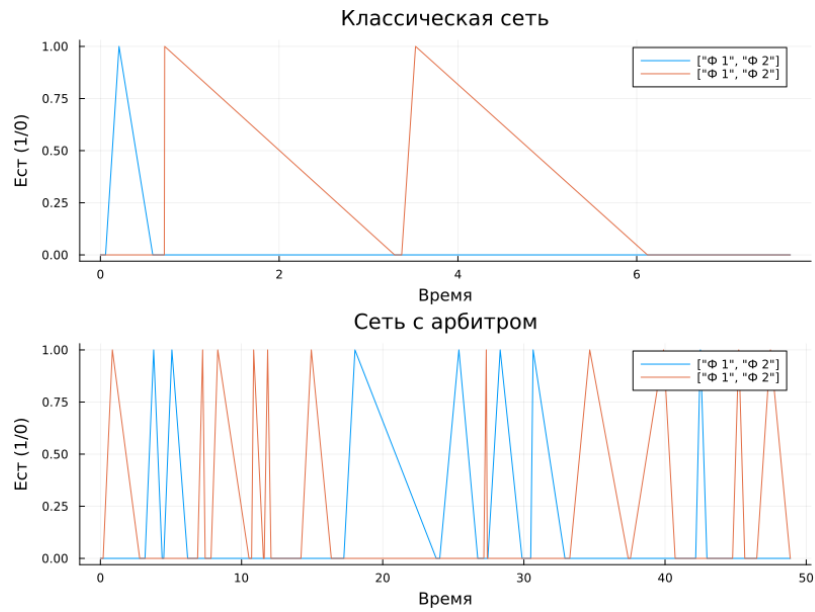


Рисунок 4.11: Код Итогового отчёта(N=2)

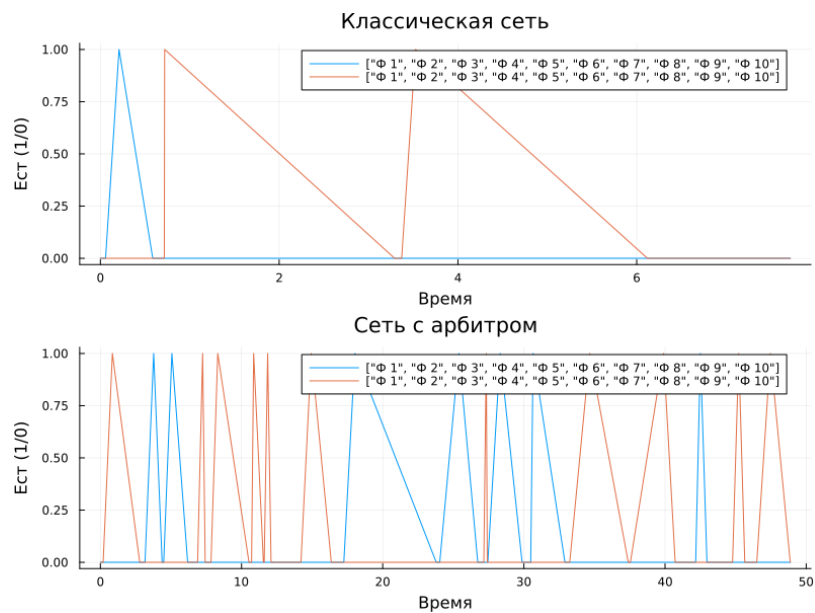


Рисунок 4.12: Код Итогового отчёта(N=10)

Я преобразовала код в литературный стиль, генерировала файл juryter и выполнила все ячейки(рис.13)

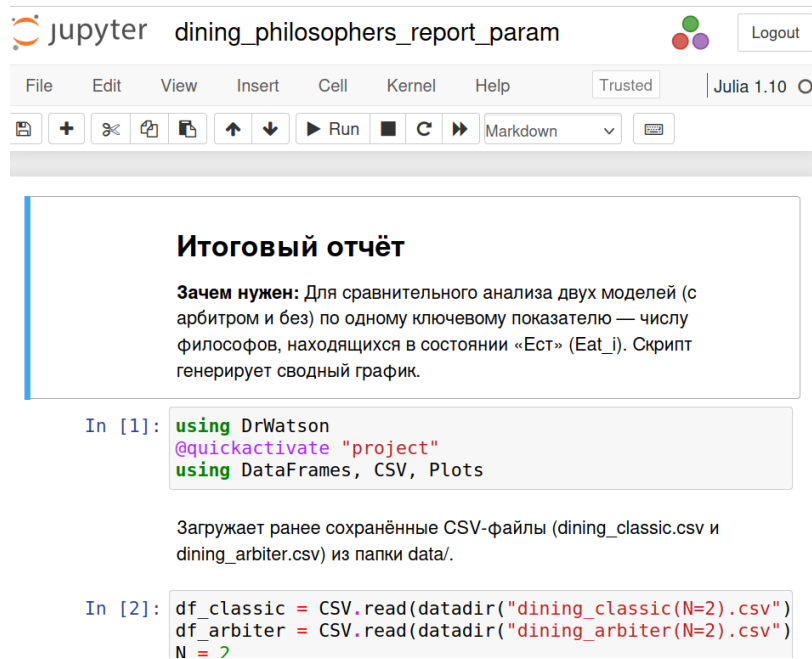


Рисунок 4.13: файл jupyter

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/scripts/dining_philosophers_report_param.qmd`

4.8 Итоговый отчёт

Зачем нужен: Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i). Скрипт генерирует сводный график.

```
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
gr(fmt=:png)
```

```
└ Warning: DrWatson could not find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
```

└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84

Plots.GRBackend()

Загружает ранее сохранённые CSV-файлы (dining_classic.csv и dining_arbiter.csv) из папки data/.

```
df_classic = CSV.read(datadir("dining_classic(N=2).csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter(N=2).csv"), DataFrame)
N = 2

df_classic1 = CSV.read(datadir("dining_classic(N=10).csv"), DataFrame)
df_arbiter1 = CSV.read(datadir("dining_arbiter(N=10).csv"), DataFrame)
N1 = 10
```

10

Столбцы для состояния «Ест»

```
eat_cols = [Symbol("Eat_$(i)") for i = 1:N]
eat_cols1 = [Symbol("Eat_$(i)") for i = 1:N1]
```

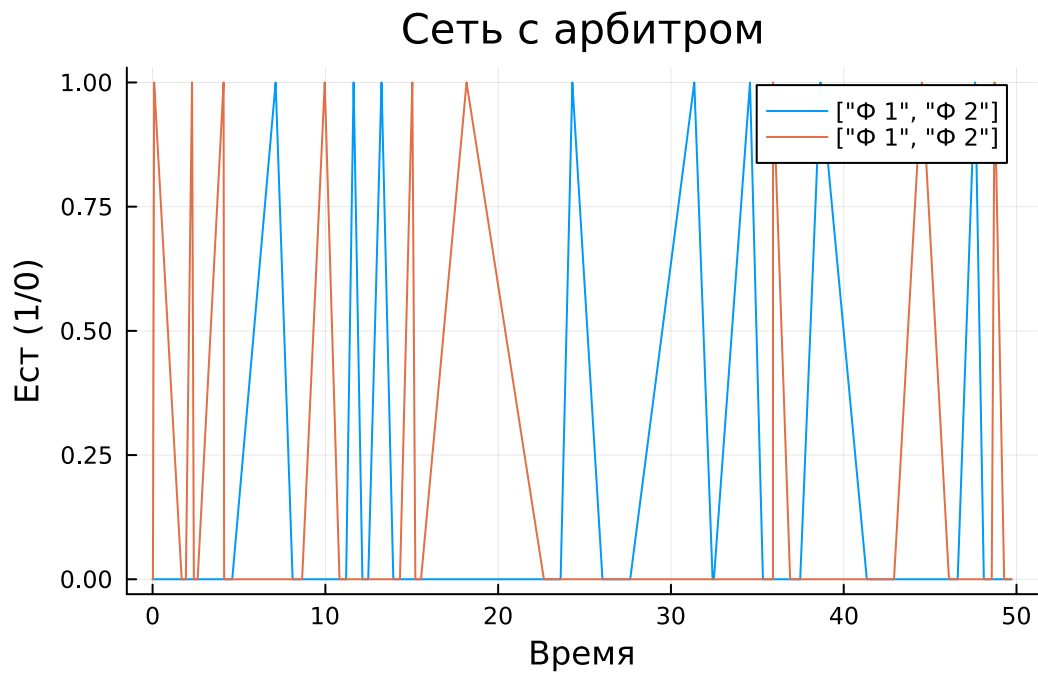
10-element Vector{Symbol}:

```
:Eat_1
:Eat_2
:Eat_3
:Eat_4
:Eat_5
:Eat_6
:Eat_7
:Eat_8
:Eat_9
:Eat_10
```

Строит два временных графика (один под другим): динамику «едящих» философов для классической сети и для сети с арбитром. $N = 2$

```
p1 = plot(
    df_classic.time,
    Matrix(df_classic[, eat_cols]),
    label = [" $\phi_i$ " for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)

p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[, eat_cols]),
    label = [" $\phi_i$ " for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
)
```



N = 10

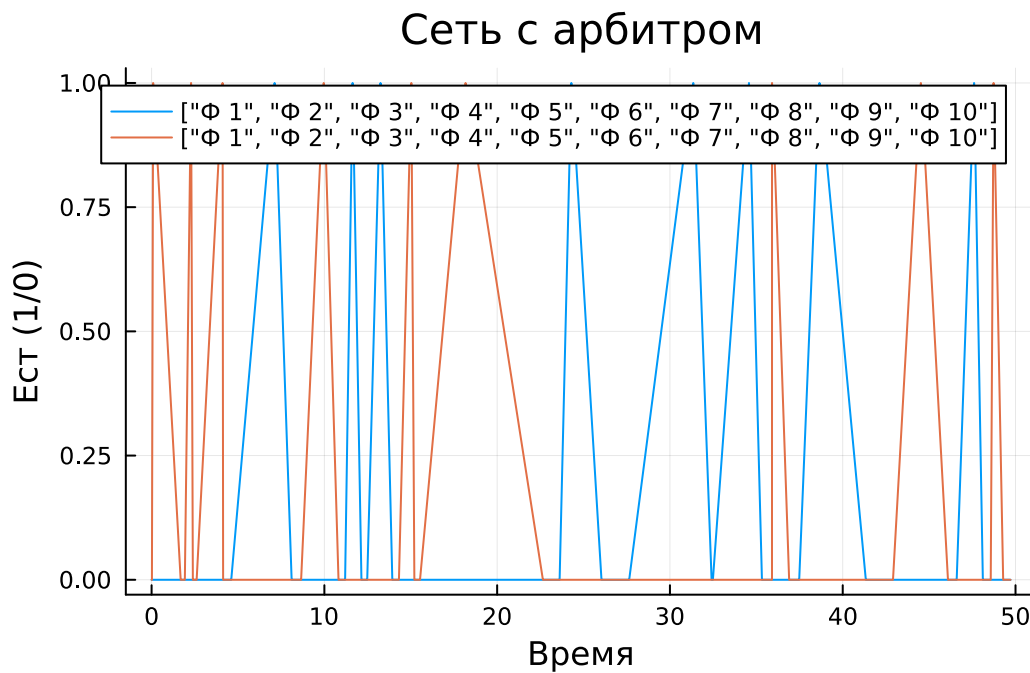
```
p3 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["Ф $i" for i = 1:N1],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)
```

```
p4 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["Ф $i" for i = 1:N1],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
```

```

title = "Сеть с арбитром",
)

```



Сохраняет объединённый график как final_report.png в папку plots/.

```

p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report(N=2).png"))

p_final1 = plot(p3, p4, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report(N=10).png"))
println("Отчёт сохранён в plots/final_report.png")

```

Отчёт сохранён в plots/final_report.png

5 Выводы

Выполняя эту лабораторную работу, научилась моделировать аппарат сетей Петри.